2023 24th IEEE International Conference on Mobile Data Management (MDM) | 979-8-3503-4101-0/23/$31.00 ©2023 IEEE | DOI: 10.1109/MDM58254.2023.00064

Situation-based Query Generation for Performance Evaluation of Cloud Managed IoT Applications

Shalmoly Mondal[1], Prem Prakash Jayaraman[1], Alireza Hassani[1], Pari Delir Haghighi[2], Dimitrios Georgakopoulos[1]
[1]*School of Science, Computing and Engineering Technologies, Swinburne University of Technology, Melbourne, Australia*
[2]*Department of Human-Centred Computing, Monash University, Melbourne, Australia*

***Abstract*—With increased deployment of IoT application on cloud platforms, assessing the performance of such application is an open problem. Currently, approaches are limited to legacy database-based applications and does not cater for the needs of IoT applications. This paper proposes, implements and validates a framework namely, IoTQGen, that can generate situation-based queries to conduct performance evaluation of IoT application hosted by cloud IoT middleware platform. The framework comprises: (i) a model to capture the query requirements of IoT applications; (ii) a data generator to generate IoT data based on specified configuration; and (iii) a set of queries designed to represent data analytic IoT applications. The framework supports different query types that can be typically used to represent and address IoT application scenarios. An important functionality of the framework is its ability to issue queries based on dynamic changes in the state of IoT entities (situations). The framework is evaluated based on two smart city use cases to highlight how the framework can be used to generate complex and dynamic queries tailored for IoT application scenarios.**

***Index Terms*—benchmarks, IoT, IoT queries, IoT middlewares, performance evaluation, cloud platforms, situations**

I. Introduction

With the wide application of IoT in various use cases, and most of them being dynamic, generation of IoT queries to represent the characteristics of IoT applications is a non-trivial task. Most of the queries have been developed for systems which have static data, and lacks the characteristics of IoT (such as dynamic and continuous data streams, heterogeneity, real-time, event-based). These hardly provide support for IoT applications. Hence, in this paper, we propose IoTQGen, a configurable framework, that allows query generation for supporting a variety of dynamic IoT scenarios.

We focused on situation based IoT applications, where entities of an IoT application such as IoT devices, and connected vehicles are involved in situations which keep on changing depending on the data generated by the entities [1]. The changing situations in turn triggers the issuing of a query. One of the key components of the framework is the query workload generator, which generates query workloads as the situation of an entity changes. The aim was to be capture and represent the dynamic changes and complexity of IoT applications and be able to generate relevant queries that are most appropriate to that situation.

A. IoT Use Case Scenario

In this section, we briefly present two smart city scenarios to highlight the challenges of dynamic IoT environment, and motivate the need for situation-based query generation to evaluate performance of IoT applications.

Traffic Monitoring and Road Safety: This application monitors the traffic conditions of a road segment and infers road incidents such as road congestion, and road blocks.

Smart Waste Collection: This application monitors road side asset and aims to identify dumped garbage on road side assets that require maintenance. It comprises trucks that collect waste across a 900 km road network. Camera sensors are mounted on these trucks to gather real-time data [2].

Based on the above use cases, we have identified the following challenges and key requirements for designing queries for IoT use cases.

Challenge 1: The data generated from sensors is not sufficient to be used directly by IoT applications. They need to be enhanced to make high-level inference or gain meaningful insights. As an example, most IoT application scenarios are associated with real-world situations, which are often represented using concepts such as entity and context spaces [3]. It has been established from the literature that sensory data combined with contextual information can help in better understanding of the environment [4]. For instance, an IoT application scenario of a smart waste disposal system involve situations that describe real-world scenarios such as: 'driving', 'idle', and 'collecting_waste'. Low level sensor data such as speed, combined with additional information such as location, and weather can be used to describe such high-level situations. Such analysis requires continuous querying over IoT data streams to infer current situations.

Challenge 2: Queries for IoT system is different from conventional database queries as IoT data is dynamic. IoT applications often involve real-time analysis (e.g., number of trucks crossing a road intersection) or analysis of historical data (e.g., average speed of trucks crossing a road segment since the last 30 minutes). Hence, IoT queries need to support a wide range of application demands, ranging from simple queries to equally complex queries. Furthermore, IoT benchmarks should include both real-time and analytical queries.

Hence, in this paper, we present IoTQgen, a framework that enables generation of IoT queries. We aim to generate a comprehensive set of queries that can be used to assess the performance of IoT applications on dynamic IoT data. The queries were designed after careful investing of the use cases discussed previously, and key features to represent such

2375-0324/23/$31.00 ©2023 IEEE
DOI 10.1109/MDM58254.2023.00064

Authorized licensed use limited to: Swinburne University of Technology. Downloaded on December 29,2023 at 06:01:30 UTC from IEEE Xplore. Restrictions apply.

queries were identified.

We summarize our contributions below:

- We propose a formal model to enable users to specify the requirements for IoT query generation for their IoT application
- We define and design a set of parameterized queries, that allows to create test configurations by changing the query parameters
- We propose a framework, IoTQGen, that incorporates the above and provides the support for automated situation based query generation

The remainder of the paper is organized as follows: Section II analyses the related work. Section III introduces IoT queries and provides a description of the query template and how the queries are modelled. Section IV introduces the proposed framework. Section V presents the evaluation of the proposed framework. Section VI concludes the paper with a road-map for future work.

II. Related Work

In order to design IoT queries for performance evaluations, the semantics of information describing the relation between data is important. In the semantic web community, there have been a few research efforts towards designing benchmark queries for the RSP systems. As an example, SRBench [5] defines a set of 17 queries covering various aspects of query processing like data integration and reasoning features over interlinked datasets. Similar studies, like LS Bench [6] and Ali et al. [7] presented a set of queries which support simple operations like aggregation, filter and few complex operations like joining historical and real-time streaming data.

However, in the IoT domain, there has been limited literature which focuses on designing such query sets. From a cloud middleware performance evaluation perspective, IoTAbench [8] is specifically for a smart meter scenario and uses simple analytical queries supporting basic aggregation operations. However, the benchmark measures query performance sequentially, unlike some benchmarks, which can measure query operations concurrently like the TPCx-IoT [9]. Moreover, the queries have been designed considering one use-case and cannot be applied to multiple scenarios, and it models one sensor data type. TPCx-IoT [9] on the other hand simulates simple time series queries which are common in IoT data analysis, but does not provide support for aggregation queries.

From the limited literature, it is evident that the existing research uses simple statistical queries, which are not sufficient to represent complex IoT application scenarios. We need to consider high-level and complex queries to address IoT specific characteristics. For instance, in a smart parking scenario application where a vehicle needs to find a suitable parking spot, if we assume the IoT entity making a query is a car, the query might have attributes like the location of the car which is varying. We need to be able to simulate or generate the change of the location in the query. More high-level queries like queries with geospatial functions, windowing functions, situation functions, etc. are required for efficient data retrieval. Most research in IoT, do not address this diversity and richness or complexity in query generation. The richness of queries can impact the overall performance of benchmarks. This is an open problem [10] one of the challenges would be generating queries that take into consideration the query richness and diversity. Hence, it is evident that traditional database queries will not suffice in creating benchmarks for IoT. The benchmark should provide queries that are versatile enough to assess the performance of IoT middleware platforms and can process dynamic IoT data.

III. Formalizing IoT Query Representation

In this section, we introduce a formal model to represent IoT queries and introduce the type of IoT queries.

A. Query Template

This section presents a template to represent and define IoT queries.

An IoT query q, can be divided into five components and can be formally represented as = $\{s^i, \varepsilon, S, o, \rho\}$, where:

- s^i is the IoT data source, denoted by $\{s^i, i \in d\}$, such that d is all the possible sensory data source: *d*=$\{$*camera sensors, speed sensors, temperature sensors,... etc.*$\}$
- ε represents all the involved entity which are being queried (e.g., vehicles, a road segment, an intersection, or a smart building)
- $S = \{S_1, S_2, \ldots, S_n\}$ represents the situations the sender of query is interested in.
- $\rho = \{\rho_1, \rho_2, \ldots, \rho_n\}$ are parameters of an IoT query such as query type, no of query instances to be issued, window size, etc.
- o represents the operators given by o=$\{$*join, filter, aggregation, enrichment, transformation*$\}$. Each of these operations are discussed in Table I.

The elements IoT data source, entity, situations, and parameter are mandatory components, while operators are optional.

The defined query template can be exemplified using the below query.

q0:Get the average speed (aggregation) of trucks (entity) in the last 15 minutes (window size) that are over speeding (situation) near a school. In this example query, the *IoT data source* is camera or speed sensors, the *entity* being queried are trucks, involved in *situation* over speeding with optional *parameter* window size of 15 minutes, and an aggregation *operation* is being performed to compute the average speed.

The above-mentioned components are captured using the proposed model discussed in the next section.

B. IoTSySML_Q

This section presents a formal model to capture the user requirements for generating IoT queries. Figure 1 illustrates the elements of the model. IoTSySML_Q is based on object-oriented concepts and uses SySML block diagram and extensions for specifying the elements [1]. This paper focuses on generating IoT queries based on the situations involved with

[1]https:sysml.org

Authorized licensed use limited to: Swinburne University of Technology. Downloaded on December 29,2023 at 06:01:30 UTC from IEEE Xplore. Restrictions apply.

Operators	Description	Example
Join	combining data from multiple data sources(IoT sensors) of the same entity or different entities	speed, temperature
Filter	filter data based on given criteria	location
Aggregation	aggregate data	sum, max, min, count, mean
Enrichment	integrating data streams to extract more information	extrapolation
Transformation	changing data to another form using transformation function	square (f=x^2)

TABLE I: Query Operators Description

IoT applications. Hence, entity, situations, and IoTQueries are the main actors of the model which are discussed here. The IoT Stack package encapsulates all objects of the IoT environment and is detailed in our previous work [1].

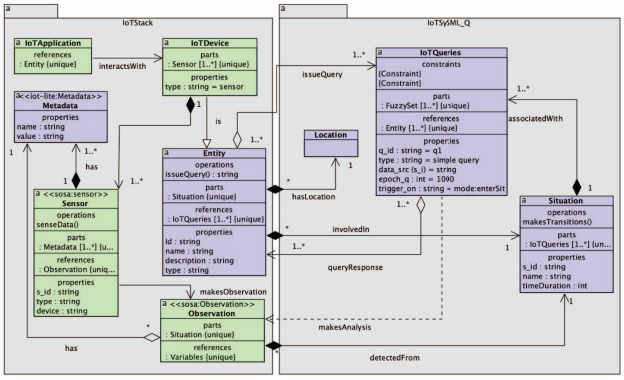


Fig. 1: Modelling Query Requirements of IoT Applications

The class *Entity* is directly associated with class *Situation*. The main functionality of this object includes issuing queries to other entities. Entities also have a class *Location* which specifies the location of the entity. Situation class describes the features and transitions that are involved in a Situation. It has attributes unique ID, name, and the time duration associated with each situation. It is also defined by the makesTransition() operation, which is a method to handle the situation transitions.

The IoTQueries class includes a unique ID attribute for each query. It contains a type attribute to indicate the type of query; the epoch_q attribute defines the number of query requests to be issued; data_src (s^i) defines the type of sensor such as speed sensors, temperature sensor, etc. Finally, a triggerOn attribute to define the rule for triggering of a query, such as issue q1 on entering situation "driving".

Queries are issued by an entity or user on entering, exiting or transitioning to a situation. For instance, if we have two situations corresponding to a traffic monitoring scenario, a vehicle entity might want to issue a Geo query to get the location of nearby vehicles every 15 mins when the situation is of "low_traffic" and the same query is issued every 10 seconds when the situation is "high_traffic". This has been illustrated in Section V-B.

After the key actors and their attributes have been defined, it is crucial to relate to the main actors to get additional information. For instance, the entity class has a property "hasLocation()", to infer the current location of the entity. This information could be obtained from the Sensor class. Each sensor instance has a unique ID attribute, and a type attribute that defines the type of sensor such as camera, speed, temperature, etc. Similarly, entities have pre-defined property "issueQuery()" which issues queries to other entities.

C. Query Classification

In this section, we introduce the classes of queries supported by our framework. Our main aim in designing the queries were to address the challenges discussed in Section I. The queries have been designed based on real world IoT use cases from two smart city application scenarios discussed in Section I-A.

1) Simple IoT Queries

We defined simple IoT queries as the ones where a single sensor data source in S^i is being queried. Simple queries can be further categorized as a *simple geo-query, simple aggregation query, or a query with windowing functionality*. Simple aggregation queries provide aggregate values(e.g., average, min, max) over a period of time. Geo-queries are used by IoT applications to retrieve data within a specific area of interest. An example of such a query is retrieving the road traffic conditions for a road intersection. Queries with windowing functionality enable controlled data access and retrieval using time slices. For this type of queries, observations are filtered based on a window [t_s, t_e], where t_s is the start time and t_e is the end time of the window. Since, IoT applications are characterized by dynamic data streams, window queries help detect changes in IoT data in real-time.

2) Complex IoT Queries

These are more complex than simple queries. These queries often involve multiple entities, ε, and operators such that $o > 1$. We define the complexity of an IoT query based on three factors: (1) involves data retrieval by combining several sensor data sources or combining sensor data with external factors to infer new knowledge; (2) inferring high-level situations from sensor data; (3) multiple query operations such as windowing, aggregation, join and filtering. Similar to simple IoT queries, complex queries can be classified as *complex aggregation query*, *complex geo-query* or *complex query with windowing operation*. For example, a query to get the truck collected the most waste in a specific area corresponds to a complex aggregation query since it includes data summarization, followed by a grouping function by truck_id and location. A further example of the different types of queries are illustrated in Section V-B.

IV. Proposed Framework - IoTQGen

The proposed framework consists of the following main components and is illustrated in Figure 2.

A. IoTSySML

This component models IoT applications which consists of IoT data and query requirements, which are taken care of

Authorized licensed use limited to: Swinburne University of Technology. Downloaded on December 29,2023 at 06:01:30 UTC from IEEE Xplore. Restrictions apply.

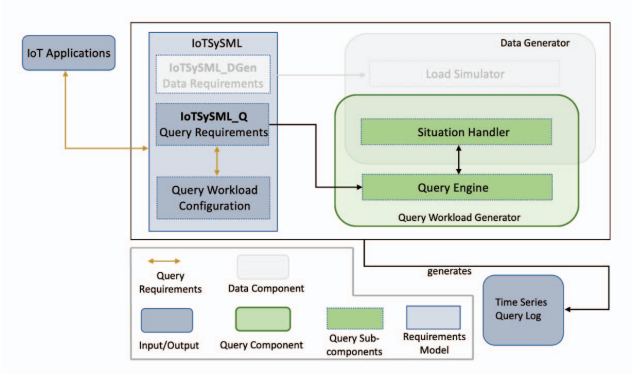


Fig. 2: High-level architecture of IoTQGen

by two separate subcomponents: IoTSySML_DGen (shown as gray faded box in Figure 2), which captures the application requirements from a data perspective [1] and IoTSySML_Q which targets the query requirements of IoT applications. The focus of this paper is IoTSySML_Q. This component consists of a query workload configuration subcomponent, which handles the configuration of the query workload parameters and contains the execution configuration for instantiating the query instances. The workload configuration parameters are use case dependent. Examples of query parameters are *epoch_q*, *situation*, *entity_no*,*sense_freq*, among others. These are discussed in Table II.

B. Query Workload Generator

This component comprises the situation handler, which is a shared component for both data and query generation components, and a query engine which generates the IoT queries.

1) Situation Handler

IoTQGen allows defining situation relevant to a particular IoT use-case and handles the situation transitions. This has been implemented using an FSI based model [1]. The algorithm used by this component entails pre-defined situations to allow situation transitions and mapping queries to entities or applications depending on the IoT use case.

2) Query Engine

This component is responsible for initializing required user inputs from IoTSySML_Q to generate IoT queries. Queries are generated based on a pre-defined query template according to the input configuration. The query engine consists of three components: a parser, which parses the user query requirements from IoTSySML_Q, a template generator which generates a template query, and a query processor. The query processor performs operations on IoT data according to the particular query operator. It is capable of performing operations like aggregation, windowing, join, and filter. Query engine is also connected to the query workload configuration subcomponent to get the configuration file for the query parameters.

V. Evaluation

Our framework is implemented as two modules: The first is the data generation module, which is implemented using the IoT data simulator [11]. The second module is the query workload generation, which is the focus of this paper. IoTQGen supports both simple and complex dynamic queries. Another novelty of the framework is that it maps IoT queries to the current situation of an entity. This section aims to evaluate the above-mentioned functionalities of the framework.

A. Dataset and Query Workloads

The data used for the experiments are generated using the IoT data simulator [11] and simulates observations from a traffic monitoring and smart waste collection IoT scenario. Data is replayed with a configured frequency of 1 minute.

All defined queries are mapped to a situation model, which entails pre-defined situations based on FSI theory [1]. Queries are issued on entering and exiting a situation. Query workloads are generated by changing the parameters values in the configuration file for a specific query type. The queries workloads are generated based on the query templates described in Section III-A. We have defined queries categorized as simple and complex queries by instantiating the parameters of the different types of queries.

Parameters	Description	Type	Example
iot_use_case	the application scenario	string	traffic-monitoring
query_type	type of query workload	string	complex queries
epoch_q	number of query requests to be generated	number	1000
data_src	type of data source	string	camera sensors
sense_freq	frequency of the incoming data	number	1000
entity_no	number of entities involved in each query	number	2
window_size	the time window	unixtime	[2022-12-09 22:56:10,2022-12-09 23:23:08]
situation	current situation	string	driving
tranistionTo	the next situation	string	idle

TABLE II: Parameters of Query Workload.

B. Case Study Based Demonstration

This section demonstrates the functionality of IoTQGen based on case studies. Due to the page limitations, we evaluate the proposed framework with 11 query examples from two real world IoT scenarios.

1) Use Case 1: Smart Waste Collection

This scenario demonstrates the framework's capability in representing queries tailored for an IoT scenario. Additionally, it also illustrates how each query is mapped to situations involved in the scenario. Here, the entities are the trucks, and the situations defined are *"driving"*, *" idle"*, and *"collecting waste"*.

q1:Get all trucks that are "idle" in the last 15 minutes

q1 exemplifies a complex IoT query where to fetch the trucks that are not driving, we first need to infer the situation

Authorized licensed use limited to: Swinburne University of Technology. Downloaded on December 29,2023 at 06:01:30 UTC from IEEE Xplore. Restrictions apply.

idle, which is achieved based on a situation function defined by the framework, and follows the format *[sensor_attr]* ϕ *[obs_value]*, where *[sensor_attr]* is the sensor data attribute, *[obs_value]* corresponds to the range of the sensor data attribute and ϕ is the comparison operator $(speed \geq 25)$. In this case, situation "idle" is defined based on the sensor attribute, speed. Then the observations are filtered based on a time window defined by [t_s, t_e].

q2: What is the average speed of a truck in the last 2 hours?

q3: What is the total waste collected by location?

q4: Get the average amount of waste collected per day, for each month, for the past year?

q2, q3, and q4 test the capability of the framework in performing aggregations and filtering observations base on a given condition. For instance, q2 performs an aggregation function (MEAN) and filters observations based on a time window, while q3 computes an aggregation function (SUM) and groups observations by location. While q2 and q3 are examples of simple aggregation queries, q4 illustrates a more complex aggregation query, since q4 first uses a subquery to calculate the total waste collected per day, using an aggregation function (SUM), and then groups observations by day. Additionally, it also requires aggregations and grouping observations by month, and finally filter observations (WHERE) made in the past year. These queries are issued depending on the current state of the entity *truck*. As an example, q2 is issued on exiting situation "S1:driving", while q5 is issued every 10 minutes to keep a check on the fuel level of the entity. Similarly, remaining queries are mapped to different situations, as illustrated in Figure 3.

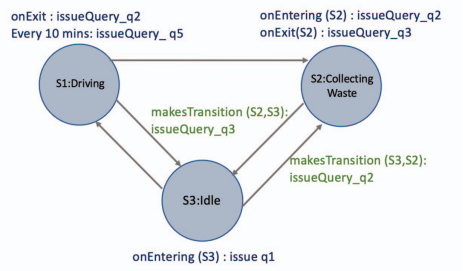


Fig. 3: Mapping IoT queries to a Smart Waste Disposal Scenario

q5: Check the fuel level of trucks every 10 minutes.

q5 exemplifies a continuous IoT query and is issued every 10 minutes on entering situation *"S1:Driving"* and hence provides a more frequent update on the status of fuel condition of a truck entity.

2) Use Case 2: Traffic Monitoring

This use case is a situation aware IoT application which identifies road incidents and alerts nearby drivers. Such scenarios have several challenges, which make the data retrieval process complicated, as discussed in Section I. One of such identified challenge is inferring high-level information from sensor data, where several additional factors are involved. As an example, a situation of "road block" can be inferred from sensor data speed and additional factors like weather can be used to determine the cause of the road block. This is an example of deriving knowledge from sensor observations as well as external factors. Another challenge is integrating data from multiple sensors. We discuss some example queries below that can be used to address the above-mentioned challenges.

q6: Fetch the last known location of a vehicle

q6 is an example of a simple geo-query, which tests the framework's capability to fetch the last known location filtered by their geospatial properties.

q7-q10 are examples of complex queries. q7 and q8 tests the engine's ability to infer new high-level knowledge from sensor observations.

q7:Notify me if there is a road congestion nearby? Similar to q1, q7 is an example of a complex IoT query and infers the situation (road congestion) from raw sensor observations (speed, no of vehicles). This query can be issued when the current situation of an entity vehicle is "idle".

q8:Get the no of vehicles which crossed the intersection and were over speeding in the last 15 minutes

q8 represents a complex IoT query, which requires combining data from multiple sensors (camera and speed sensors) and a window operation. The query requires continuous monitoring of a specific intersection to compute the number of vehicles crossing an intersection, that were *over speeding* in a 15-minute window from current time. The query uses the situation function operator to determine the current situation of over speeding, which can be defined as an event which arises when entities exceed the speed limit.

q9 and q10 are examples of complex aggregation queries and tests the framework's ability to generate and execute complex aggregations and sub queries.

q9: What is the peak travel time on a freeway and the estimated travel time per hour of a day to my destination?

This query aggregates the travel time on a specific type of road (freeway) by hour of the day. It uses aggregation functions like MAX, MIN and AVG to compute the maximum, minimum and average estimated travel times. The observations are further filtered by day and road type using a WHERE clause, and are grouped by hour through the GROUP BY clause. q8 can be used to optimize traffic by implementing variable speed limits or adjusting traffic signals.

q10:Get the intersections which has observed peak traffic in the last 2 hours.

The above query also exemplifies a complex aggregation query, which involves execution of subqueries. The aim of q10 is to identify intersections that have observed peak traffic in the last 2 hours. The maximum traffic is computed using the MAX aggregation function within a subquery. The result of the subquery is used to filter observations to include data for only two hours. The results are then grouped by intersection ID using the GROUP BY clause, and ORDER BY clause is used to order the sensor observations in descending order. Finally, the top 10 intersections with the most sensor values are returned

Authorized licensed use limited to: Swinburne University of Technology. Downloaded on December 29,2023 at 06:01:30 UTC from IEEE Xplore. Restrictions apply.

using the LIMIT clause. q10 is issued on entering the situation "driving". Figure 4 illustrates this query.

```
SELECT intersection_id, COUNT(*) as no_obs FROM traffic_data
WHERE device_id = "sen02" AND timestamp >= NOW() - INTERVAL '2 hours'
AND traffic_volume = (SELECT MAX(traffic_volume) FROM traffic_data
WHERE timestamp >= NOW() - INTERVAL '2 hours')AND situation = S1
GROUP BY intersection_id ORDER BY no_obs
DESC LIMIT 10;
```

Fig. 4: Illustration of a complex aggregation query

q11:Fetch the percentage of vehicles using alternative routes to avoid traffic congestion during peak hour and the weather conditions.

This query demonstrates the framework's ability to integrate sensor observations with external factors such as weather observations to obtain results. It joins traffic data with weather data to analyze the effect of weather condition during peak hours.

This work mainly focused on a functional evaluation, however, we propose metrics that can be used for performance evaluation using IoTQGen. *Query response time:* The time taken to execute a query after it is triggered on detection of an event. The query response time is effected by varying data ingestion rate and the complexity of the queries; *Scalability:* how the system responds to multiple instances of query requests (multiple applications or entities issuing query requests); *Throughput:* the maximum number of query requests that can be handled per unit time.

Additionally, interoperability requirements for data providers can play a critical role in the overall performance of an IoT query engine. It is crucial to consider the data transfer formats and protocols to ensure efficiency and reliability in retrieving information. Without proper interoperability between different providers, the query engine may fail to retrieve the needed information. Establishing a standard data model can address these issues and promote effectively data retrieval to support complex queries.

VI. Conclusion

In this paper, we proposed, implemented and evaluated IoTQGen, a novel framework for efficient situation-based generation of IoT queries to evaluate cloud-middleware hosted IoT applications. It consists of two key components: IoTSySML_Q for capturing the query requirements of various IoT applications, and a query workload generator which generated both simple and complex queries. We demonstrated that the framework was able to generate queries that are representative of real-world IoT scenarios via examples of two smart city use-cases. We plan to extend our framework, to design an algorithm to map keywords from natural language queries to create IoT application queries. In addition, we also plan to develop a standard data model to facilitate interoperability between different data providers, leading to improved data retrieval. These efforts aim to enhance the functionality and efficiency of our system, enabling better integration and utilization of IoT data.

References

[1] S. Mondal, P. P. Jayaraman, P. Delir Haghighi, A. Hassani, and D. Georgakopoulos, "Situation-Aware IoT Data Generation towards Performance Evaluation of IoT Middleware Platforms," *Sensors*, vol. 23, no. 1, p. 7, Jan. 2023, number: 1 Publisher: Multidisciplinary Digital Publishing Institute. [Online]. Available: https://www.mdpi.com/1424-8220/23/1/7

[2] A. R. M. Forkan, Y.-B. Kang, F. Marti, S. Joachim, A. Banerjee, J. K. Milovac, P. P. Jayaraman, C. McCarthy, H. Ghaderi, and D. Georgakopoulos, "Mobile IoT-RoadBot: an AI-powered mobile IoT solution for real-time roadside asset management," in *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking*, ser. MobiCom '22. New York, NY, USA: Association for Computing Machinery, Oct. 2022, pp. 883–885. [Online]. Available: https://doi.org/10.1145/3495243.3558271

[3] A. Padovitz, S. Loke, and A. Zaslavsky, "Towards a theory of context spaces," in *IEEE Annual Conference on Pervasive Computing and Communications Workshops, 2004. Proceedings of the Second*, Mar. 2004, pp. 38–42.

[4] N. Kolbe, A. Zaslavsky, S. Kubler, J. Robert, and Y. Le Traon, "Enriching a Situation Awareness Framework for IoT with Knowledge Base and Reasoning Components," May 2017. [Online]. Available: https://publications.csiro.au/rpr/pub?pid=csiro:EP17955

[5] Y. Zhang, P. M. Duc, O. Corcho, and J.-P. Calbimonte, "SRBench: A Streaming RDF/SPARQL Benchmark," in *The Semantic Web – ISWC 2012*, ser. Lecture Notes in Computer Science, P. Cudré-Mauroux, J. Heflin, E. Sirin, T. Tudorache, J. Euzenat, M. Hauswirth, J. X. Parreira, J. Hendler, G. Schreiber, A. Bernstein, and E. Blomqvist, Eds. Berlin, Heidelberg: Springer, 2012, pp. 641–657.

[6] MERLOT, "The Linked Sensor Middleware - Connecting the real world and the Semantic Web." [Online]. Available: https://www.merlot.org/merlot/viewMaterial.htm?id=967590

[7] M. I. Ali, F. Gao, and A. Mileo, "CityBench: A Configurable Benchmark to Evaluate RSP Engines Using Smart City Datasets," in *The Semantic Web - ISWC 2015*, ser. Lecture Notes in Computer Science, M. Arenas, O. Corcho, E. Simperl, M. Strohmaier, M. d'Aquin, K. Srinivas, P. Groth, M. Dumontier, J. Heflin, K. Thirunarayan, and S. Staab, Eds. Cham: Springer International Publishing, 2015, pp. 374–389.

[8] M. Arlitt, M. Marwah, G. Bellala, A. Shah, J. Healey, and B. Vandiver, "IoTAbench: an Internet of Things Analytics Benchmark," in *Proceedings of the 6th ACM/SPEC International Conference on Performance Engineering*, ser. ICPE '15. New York, NY, USA: Association for Computing Machinery, Jan. 2015, pp. 133–144. [Online]. Available: https://doi.org/10.1145/2668930.2688055

[9] M. Poess, R. Nambiar, K. Kulkarni, C. Narasimhadevara, T. Rabl, and H.-A. Jacobsen, "Analysis of TPCx-IoT: The First Industry Standard Benchmark for IoT Gateway Systems," in *2018 IEEE 34th International Conference on Data Engineering (ICDE)*, Apr. 2018, pp. 1519–1530, iSSN: 2375-026X.

[10] A. Hassani, A. Medvedev, P. Delir Haghighi, S. Ling, A. Zaslavsky, and P. Prakash Jayaraman, "Context Definition and Query Language: Conceptual Specification, Implementation, and Evaluation," *Sensors*, vol. 19, no. 6, p. 1478, Jan. 2019, number: 6 Publisher: Multidisciplinary Digital Publishing Institute. [Online]. Available: https://www.mdpi.com/1424-8220/19/6/1478

[11] "IBA-Group-IT/IoT-data-simulator," Feb. 2023, original-date: 2018-04-09T14:02:53Z. [Online]. Available: https://github.com/IBA-Group-IT/IoT-data-simulator

Authorized licensed use limited to: Swinburne University of Technology. Downloaded on December 29,2023 at 06:01:30 UTC from IEEE Xplore. Restrictions apply.